

Pre-requisites

Story Duration: 1h 0m

Review

SA

Assigned to Saravanan S

Description

Python 3.10+

Official Documentation: [Google AI for Developers](#)

Installation Guide: [Tom's Hardware](#)

Popular Tutorial: [Python Programming Tutorial - Full Course for Beginners](#)

Installation Steps:

Download the latest Python 3.10+ installer for your operating system from the official website.

Run the installer and ensure you check the box that says "Add Python to PATH".

Follow the installation prompts to complete the setup.

Verify the installation by opening a terminal or command prompt and typing `python --version`.

FastAPI Framework –

Official Documentation: [FastAPI](#)

User Guide: [FastAPI](#)

Popular Tutorial: [FastAPI Crash Course](#)

HTML & CSS – Basic templating used in /templates and /static

Google Gemini API Key

Official Documentation: [Google AI for Developers](#)

Setup Guide: [GeeksforGeeks](#)

Popular Tutorial: [How to Use Google Gemini API Key](#)

Setup Steps:

Visit the Google AI Studio and sign in with your Google account.

Create a new project and enable the Gemini API.

Generate an API key and securely store it.

Uvicorn (ASGI Server)

Official Documentation: [PyPI](#)

Installation Steps:

Install Uvicorn using pip: `pip install uvicorn`

Jinja2 (HTML Templating Engine)

Official Documentation: [Jinja2 Documentation](#)

Installation Steps:

Install Jinja2 using pip: `pip install jinja2`

Workflow

Story Duration: 1h 0m

To Do

TH

Assigned to Theopin B

Description

Epic 1: Model Selection and Architecture

Select AI Models

Epic 2: Core Functionalities Development

Module Implementation

Backend API with FastAPI

Epic3: Frontend Development

Build Web Interface

Live Integration

Epic 4: Testing and Deployment

Run Locally

Functional Testing

Select AI Models

Story Duration: 1h 0m

To Do

TH

Assigned to Theopin B

Description

Ai Models:

Gemini 1.5 Pro (via API):

Used for: Q&A, summarization, quiz generation, learning paths

Benefits: Advanced reasoning, structured outputs, and cloud inference

LaMini-Flan-T5-783M (local):

Used for: Concept explanation

Benefits: Instruction-tuned, lightweight, CPU-compatible

Folder Architecture:

EduGenie/	
main.py	# FastAPI app
explanation_ module.py	# Concept explanation logic
qna.py	# Question answering
quiz_ module.py	# Quiz generation
summary_ module.py	# Summarization
learning_ path.py	# Learning recommendations
templates/index.html	# HTML frontend
static/style.css	# Styling
requirements.txt	# Python dependencies

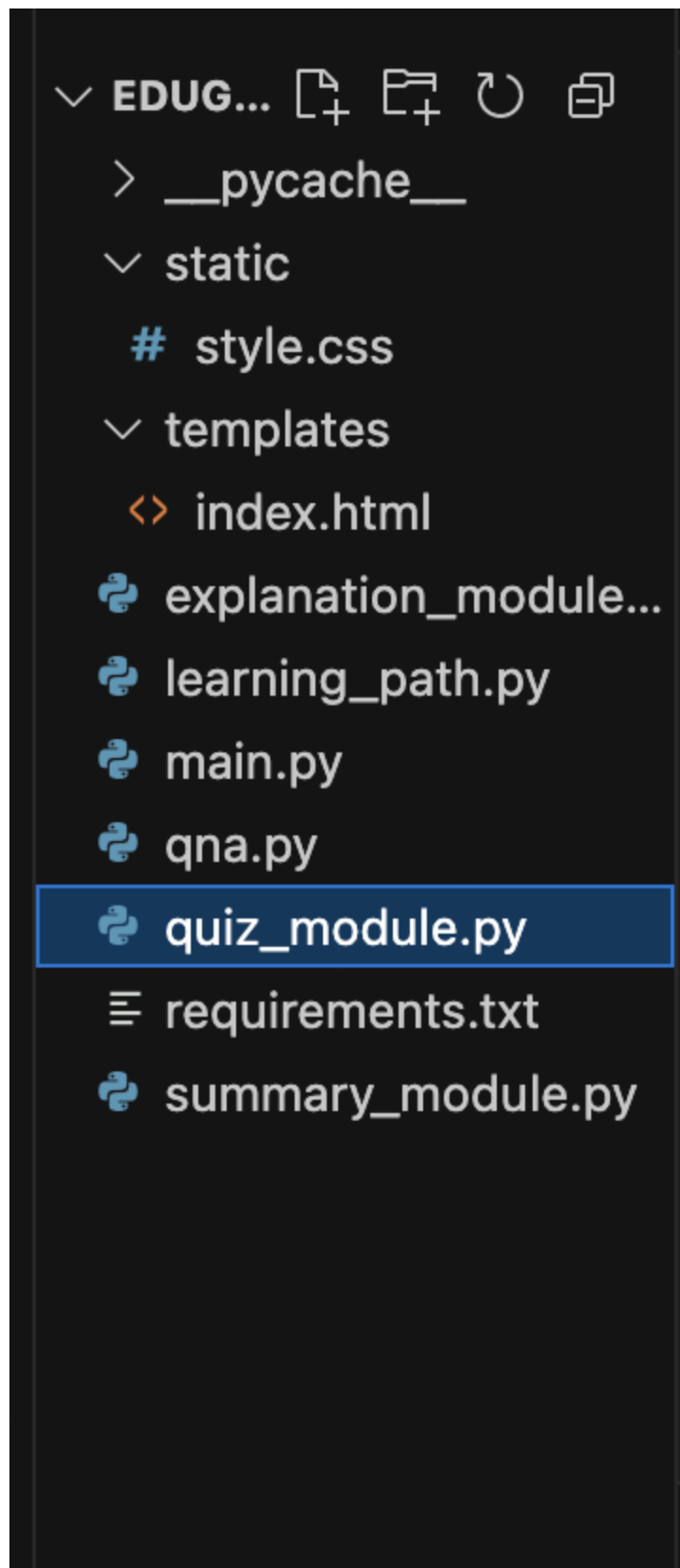


FIG. Showing the Folder

Architecture

Module Implementation

Story Duration: 1h 30m

To Do

AN

Assigned to Antony Ragul A

Description

Explanation Module

EduGenie utilizes the LaMini-Flan-T5 model, a lightweight yet powerful generative AI, to deliver educational content in a simplified and highly readable manner. This model is specifically fine-tuned to provide concise, context-aware responses that break down complex topics into easily understandable language. By focusing on clarity and brevity, EduGenie ensures that learners with minimal background knowledge or technical experience can grasp essential concepts without feeling overwhelmed. This makes it particularly valuable for beginners, school students, or self-learners seeking straightforward explanations. The integration of LaMini-Flan-T5 empowers EduGenie to act as a reliable and accessible study companion for foundational learning across subjects.

```
planation_module.py
from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Load the improved model for explanations
explain_tokenizer = AutoTokenizer.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")
explain_model = AutoModelForSeq2SeqLM.from_pretrained("MBZUAI/LaMini-Flan-T5-783M")

def explain_topic(topic: str) -> str:
    input_text = f"Explain the concept of '{topic}' in a simple and clear way for a school student."
    inputs = explain_tokenizer(input_text, return_tensors="pt")

    outputs = explain_model.generate(
        **inputs,
        max_new_tokens=150,
        temperature=0.7,
        top_k=50,
        top_p=0.95,
        do_sample=True
    )

    explanation = explain_tokenizer.decode(outputs[0], skip_special_tokens=True)
    return explanation
```

QnA Module

EduGenie is powered by Gemini 1.5 Pro, enabling it to handle a wide range of general knowledge and academic question-answering tasks with precision. Its advanced understanding and contextual capabilities make it an effective tool for learners seeking accurate, AI-driven assistance across diverse educational topics and subjects.

```
def answer_question_with_gemini(question: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        response = model.generate_content(question)
        return response.text.strip()
    except Exception as e:
        return f"⚠ Error in QnA: {e}"
```

Quiz Module

EduGenie generates three multiple-choice questions (MCQs) from a given passage, each containing four carefully crafted options. It utilizes the Gemini model to comprehend the context and meaning of the passage, ensuring that the questions are relevant and the distractors are plausible. The final output is structured in JSON format for easy integration.

```

def clean_json_block(text):
    # Remove Markdown ```json code fences
    return re.sub(r"```(?:json)?\n(.*?)```", r"\1", text, flags=re.DOTALL).strip()

def generate_quiz(text: str) -> list:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")

        prompt = f"""
You are a quiz generator.

From the following passage, create 3 multiple-choice questions. Each question should include:
- A "question"
- A list of 4 "options"
- A correct "answer" that must exactly match one of the options.

Format your output as **valid JSON**, like this:
[
  {{
    "question": "What is ...?",
    "options": ["A", "B", "C", "D"],
    "answer": "A"
  }}
]

Passage:
{text}
"""

        response = model.generate_content(prompt)
        quiz_text = response.text.strip()

        # 🧹 Clean markdown code blocks if any
        cleaned_text = clean_json_block(quiz_text)

```

It sends a structured prompt to the model, expecting a valid JSON response with each question containing four options and a correct answer. To ensure proper parsing, it cleans any Markdown code blocks using the `clean_json_block` function. The cleaned response is then parsed into a Python list. If an error occurs during generation or parsing, it returns a detailed error message for debugging.

Summary Module

This feature leverages Gemini's generative capabilities to summarize long paragraphs into concise, easy-to-understand versions, making it ideal for quick revision. It ensures the core information is retained while eliminating redundancy, helping learners absorb key points efficiently without losing clarity or context.

```
def summarize_text(text: str) -> str:
    try:
        model = genai.GenerativeModel(model_name="models/gemini-1.5-pro")
        prompt = f"Summarize the following text in simple language:\n\n{text}"
        response = model.generate_content(prompt)
        return response.text.strip()
    except Exception as e:
        return f"⚠️ Error in Summary: {e}"
```

Learning Path Module

The `get_learning_recommendations` function uses Google Gemini to generate a personalized, structured learning path for any given topic. By crafting a detailed prompt, it instructs the AI to suggest beginner to advanced concepts, organized by difficulty, and supported with useful resources such as videos, articles, or books. It ensures adaptability to the learner's level. The function safely handles API responses and errors, providing either the generated text or a helpful error message if the response is invalid or incomplete.

```
def get_learning_recommendations(topic):
    prompt = f"""
    You are an AI tutor. The student wants to learn about: {topic}.
    Suggest a structured and adaptive learning path including key topics, order of learning, and resources (
    Include beginner, intermediate, and advanced levels if needed.
    """
    try:
        response = model.generate_content(prompt)
        print("🧠 Gemini raw response:", response)

        if hasattr(response, "text"):
            return response.text
        elif hasattr(response, "parts") and response.parts:
            return response.parts[0].text
        else:
            return "❌ Could not extract content from Gemini response."
    except Exception as e:
        import traceback
        traceback.print_exc() # ✅ This gives full error trace
        return f"❌ Error occurred: {str(e)}"
```


Backend API with FastAPI

Story Duration: 1h 30m

To Do

SA

Assigned to Saravanan S

Description

Defined RESTful endpoints for each module

/qa

/explain

/quiz

/summarize

/learn/recommendations

Connected each endpoint to a respective module logic

```
main.py
40
41 # Q&A - GET API using Gemini
42 @app.get("/qa")
43 async def answer_question(question: str = Query(...)):
44     answer = answer_question_with_gemini(question)
45     return {"answer": answer}
46
47 # Explanation - POST API
48 @app.post("/explain/")
49 async def explain_api(request: Request):
50     data = await request.json()
51     topic = data.get("topic")
52     if not topic:
53         return JSONResponse(content={"error": "Please provide a topic."}, status_code=400)
54     explanation = explain_topic(topic)
55     return {"topic": topic, "explanation": explanation}
56
57 # Summarization - POST API
58 @app.post("/summarize/")
59 async def summarize_api(request: Request):
60     data = await request.json()
61     text = data.get("text")
62     if not text:
63         return JSONResponse(content={"error": "Please provide text to summarize."}, status_code=400)
64     summary = summarize_text(text)
65     return {"summary": summary}
66
```

```

66
67 # Quiz Generation - POST API
68 @app.post("/quiz")
69 async def quiz_api(request: Request):
70     data = await request.json()
71     text = data.get("text")
72     if not text:
73         return JSONResponse(content={"error": "Please provide text for quiz."}, status_code=400)
74     quiz = generate_quiz(text)
75     print("Generated quiz:", quiz) # ✓ DEBUG
76     return JSONResponse(content={"quiz": quiz})
77
78 # Learning Recommendations - GET API
79 @app.get("/learn/recommendations")
80 async def learning_recommendation_api(topic: str = Query(...)):
81     recommendation = get_learning_recommendations(topic)
82     return {"topic": topic, "recommendation": recommendation}
83
84

```

Fig. Endpoints of API

Build Web Interface

Story Duration: 1h 30m

To Do

AN

Assigned to Antony Ragul A

Description

HTML Form (index.html):

Task dropdown (Explain, QnA, Quiz, Summary, Recommend Path)

Text area for user input

Submit button

CSS Styling (style.css):

Responsive design

Styled buttons, input areas, and result container

```

templates > <> index.html > ...
 2  <html lang="en">
 3  <head>
86  </head>
87  <body>
88      <h1>Welcome to EduGenie 🤖!</h1>
89      <p>Your personal AI tutor for learning support!</p>
90
91      <!-- Q&A -->
92      <form id="qaForm">
93          <label><strong>Ask EduGenie a Question:</strong></label><br><br>
94          <input type="text" id="question" placeholder="Why is the sky blue?" required>
95          <button type="submit">Get Answer</button>
96      </form>
97      <div id="qaResult" class="output"></div>
98
99      <!-- Explanation -->
100     <form id="explainForm">
101         <label><strong>Need an Explanation:</strong></label><br><br>
102         <input type="text" id="topic" placeholder="Photosynthesis" required>
103         <button type="submit">Explain</button>
104     </form>
105     <div id="explanationResult" class="output"></div>
106
107     <!-- Summary -->
108     <form id="summaryForm">
109         <label><strong>Summarize a Paragraph:</strong></label><br><br>
110         <input type="text" id="summaryText" placeholder="Paste long content to summarize" required>
111         <button type="submit">Summarize</button>
112     </form>
113     <div id="summaryResult" class="output"></div>
114
115     <!-- Quiz -->
116     <form id="quizForm">
117         <label><strong>Generate a Quiz:</strong></label><br><br>
118         <input type="text" id="quizText" placeholder="Solar System" required>
119         <button type="submit">Generate Quiz</button>
120     </form>
121     <div id="quizResult" class="output"></div>

```

Fig. Showing Frontend Intergration

Live Integration

Story Duration: 1h 30m

To Do

AN

Assigned to Antony Ragul A

Description

Form submission sends POST request to FastAPI backend

Result appears below input box in real-time

Run Locally

Story Duration: 1h 30m

To Do

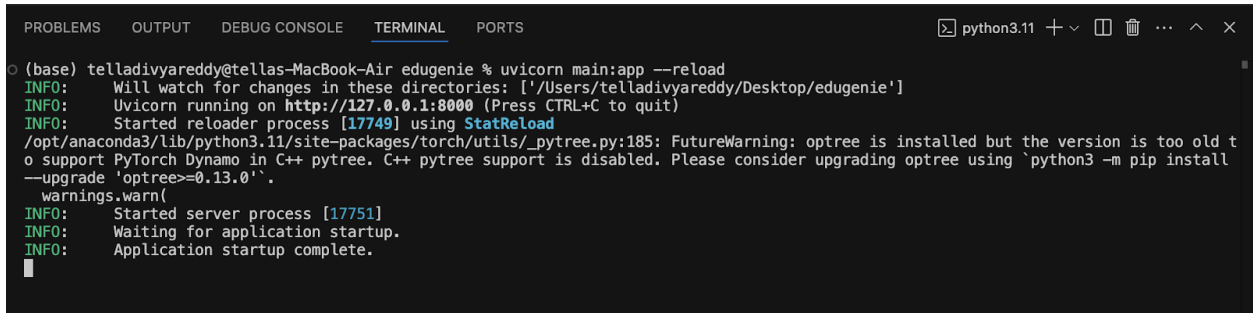
KA

Assigned to Karthick M

Description

Run locally with `uvicorn main:app --reload`

Navigate to: <http://127.0.0.1:8000>

A terminal window with a dark background and light-colored text. The window has tabs at the top labeled 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'TERMINAL' (which is active), and 'PORTS'. The terminal shows the command `(base) telladivyaireddy@tellas-MacBook-Air edugenie % uvicorn main:app --reload` and its output. The output includes several INFO messages: 'Will watch for changes in these directories: [...]', 'Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)', 'Started reloader process [17749] using StatReload', and 'Started server process [17751]'. There is also a FutureWarning about optree version and a message 'Waiting for application startup.' followed by 'Application startup complete.'.

```
(base) telladivyaireddy@tellas-MacBook-Air edugenie % uvicorn main:app --reload
INFO:      Will watch for changes in these directories: ['/Users/telladivyaireddy/Desktop/edugenie']
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO:      Started reloader process [17749] using StatReload
/opt/anaconda3/lib/python3.11/site-packages/torch/utils/_pytree.py:185: FutureWarning: optree is installed but the version is too old to support PyTorch Dynamo in C++ pytree. C++ pytree support is disabled. Please consider upgrading optree using `python3 -m pip install --upgrade 'optree>=0.13.0'`.
  warnings.warn(
INFO:      Started server process [17751]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
```

Fig. Showing the Application can be accessed

Functional Testing

Story Duration: 1h 30m

To Do

TH

Assigned to Theopin B

Description

Ask a question

Get an explanation

Generate a quiz

Summarize content

Get a personalized learning plan

Welcome to EduGenie 🧠✨

Your personal AI tutor for learning support!

Ask EduGenie a Question:

Need an Explanation?

Summarize a Paragraph:

Fig. EDUGENIE

Summarize a Paragraph:

Generate a Quiz:

Get Learning Recommendations:

Exploring EduGenie:

a. Asking questions:

Welcome to EduGenie 🧠✨

Your personal AI tutor for learning support!

Ask EduGenie a Question:

Get Answer

Answer:

The Pacific Ocean is the largest ocean on Earth.

b. Explanation of any topic:

Need an Explanation?

Explain

Explanation:

Quantum computing is a type of computing that uses the principles of quantum mechanics to perform certain calculations much faster than traditional computers. It allows computers to perform certain calculations much faster than traditional computers because it uses tiny particles called qubits that can exist in multiple states at the same time. This allows for new kinds of calculations and solving problems that are not possible with traditional computers.

Need an Explanation?

Explain

Explanation:

The binary search algorithm is a way to find the index of a target value in a sorted list of numbers. It's like searching for a needle in a haystack. You need to find the number between 0 and 1, and then go back to the beginning of the list to find the target value.

c. Summarising long paragraphs:

Summarize a Paragraph:

Summarize

Summary:

The Industrial Revolution changed the world by shifting it from farming to factories. New machines, like James Watt's steam engine, made things faster and easier to produce. While this boosted economies, it also created problems like bad working conditions, child labor, pollution, and overcrowded cities. Even with these problems, the Industrial Revolution shaped the modern world, affecting how we travel, communicate, work, and trade.

d. Generating quizzes:

It generates three questions with 4 options each. It corrects the answer if chosen wrong.

Generate a Quiz:

Generate Quiz

Quiz:

Q1: What does the Pythagorean theorem describe?

- ☐ The relationship between the angles of a triangle.
- ☐ The relationship between the sides of a right-angled triangle.
- ☐ The relationship between the area and perimeter of a triangle.
- ☐ The relationship between the sides of any triangle.

Check Answer

Quiz:

Q1: What does the Pythagorean theorem describe?

- ☐ The relationship between the angles of a triangle.
- ☒ The relationship between the sides of a right-angled triangle.
- ☐ The relationship between the area and perimeter of a triangle.
- ☐ The relationship between the sides of any triangle.

Check Answer

✓ Correct!

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

- ☐ $a + b = c$

Q2: If 'a' and 'b' are the lengths of the two shorter sides of a right-angled triangle, and 'c' is the length of the longest side (hypotenuse), what equation represents the Pythagorean theorem?

- ☐ $a + b = c$
- ☐ $a^2 + b^2 = c^2$
- ☒ $a^2 - b^2 = c^2$
- ☐ $2a + 2b = 2c$

Check Answer

✗ Incorrect. Correct answer: $a^2 + b^2 = c^2$

Q3: Which type of triangle does the Pythagorean theorem apply to?

- ☐ Equilateral triangles
- ☐ Isosceles triangles
- ☒ Right-angled triangles
- ☐ All types of triangles

Check Answer

✓ Correct!

e. Guiding with learning recommendations from beginners level to advanced level

Get Learning Recommendations:

SQL

Get Recommendations

Learning Recommendations for "SQL":

SQL Learning Path: From Zero to Hero

This learning path is structured to progressively introduce SQL concepts, starting from the basics and gradually advancing to more complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

****I. Beginner Level: Building a Foundation****

****(Estimated Time: 1-2 weeks)****

*** **Key Topics:****

- * What is a Database and SQL? (Relational Model, DBMS)
- * Basic Syntax (SELECT, FROM, WHERE)
- * Data Types (INT, VARCHAR, DATE, etc.)
- * Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)
- * Ordering Results (ORDER BY)
- * Limiting Results (LIMIT/OFFSET or TOP)
- * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)

complex topics. It's designed to be adaptive: feel free to adjust the pace and delve deeper into areas that particularly interest you.

****I. Beginner Level: Building a Foundation****

****(Estimated Time: 1-2 weeks)****

*** **Key Topics:****

- * What is a Database and SQL? (Relational Model, DBMS)
- * Basic Syntax (SELECT, FROM, WHERE)
- * Data Types (INT, VARCHAR, DATE, etc.)
- * Filtering Data (WHERE clause with comparison operators, logical operators: AND, OR, NOT)
- * Ordering Results (ORDER BY)
- * Limiting Results (LIMIT/OFFSET or TOP)
- * Basic Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
- * Introduction to Tables and Columns

*** **Resources:****

- * ****Interactive Tutorials:****
 - * SQLZoo: Excellent for hands-on practice with different SQL flavors.
 - * Khan Academy's SQL course: Offers a good introductory overview.
 - * Codecademy's Learn SQL: Interactive lessons with practical exercises.
- * ****Videos:****
 - * FreeCodeCamp's SQL Tutorial: Comprehensive and beginner-friendly.

****II. Intermediate Level: Working with Multiple Tables****

****(Estimated Time: 2-3 weeks)****


```

**II. Intermediate Level: Working with Multiple Tables**

**(Estimated Time: 2-3 weeks)**

* **Key Topics:**
  * Joining Tables (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN)
  * Subqueries (Nested queries)
  * Grouping Data (GROUP BY)
  * Filtering Groups (HAVING)
  * Set Operations (UNION, INTERSECT, EXCEPT)
  * Working with Strings (string concatenation, substrings, LIKE operator)
  * Working with Dates and Times (date formatting, date arithmetic)
  * Views (Creating and using views)

* **Resources:**
  * **Books:**
    * "SQL Queries for Mere Mortals" by Michael J. Hernandez: A great resource for understanding practical SQL querying.
  * **Online Documentation:** Refer to the documentation for your specific SQL database (e.g., MySQL, PostgreSQL, SQL Server).
  * **Practice Platforms:** LeetCode, HackerRank, and similar platforms offer SQL challenges to reinforce your skills.

**III. Advanced Level: Mastering Database Management**

**(Estimated Time: 3-4 weeks and beyond)**

* **Key Topics:**
  * Stored Procedures and Functions
  * Triggers
  * Indexes and Performance Tuning
  * Transactions and Concurrency Control
  * Database Design (Normalization)

  * **Books:**
    * "SQL Performance Explained" by Markus Winand: In-depth coverage of SQL performance optimization.
    * "Database Internals" by Alex Petrov: Explains the inner workings of database systems.
  * **Advanced Online Courses:** Look for specialized courses on database administration and performance tuning on platforms like Udemy and Coursera.
  * **Community Forums and Blogs:** Engage with the SQL community to learn from experienced professionals and stay up-to-date with the latest advancements.

**Adaptive Learning Tips:**

* **Start with the basics:** Don't rush into advanced topics before mastering the fundamentals.
* **Practice regularly:** The more you practice writing SQL queries, the better you'll become.
* **Use real-world datasets:** Working with real data will help you understand how SQL is used in practical scenarios.
* **Focus on understanding the concepts:** Don't just memorize syntax; understand the underlying logic behind SQL queries.
* **Break down complex problems:** Divide large problems into smaller, manageable parts.
* **Don't be afraid to experiment:** Try different approaches and see what works best.
* **Seek help when needed:** Don't hesitate to ask for assistance from online communities, forums, or mentors.

This learning path provides a solid foundation for mastering SQL. Remember to adapt it to your own learning style and goals, and enjoy the journey of becoming proficient in this powerful language!

```

It provides along with details of resources from where to learn and stepwise guidance through-out.

Conclusion

EpicComplexity: easyDuration: 3h 0m

Description

EduGenie has been developed as a robust and accessible AI-powered educational assistant that seamlessly integrates cloud-based intelligence with an intuitive, low-footprint web interface. Designed to support self-learners, educational institutions, and content platforms, EduGenie democratizes learning by simplifying complex concepts, providing instant question-and-answer interactions, generating personalized quizzes, offering detailed summaries, and tailoring smart learning paths based on individual needs. Its lightweight infrastructure ensures that even users with minimal hardware resources can access high-quality, personalized education without barriers. Throughout the development of EduGenie, the project achieved key milestones in AI integration, user experience design, and content adaptability. Leveraging advanced generative AI, it transforms static learning into a dynamic and interactive process. The platform's simplicity and clarity cater to learners of all levels—especially those who may find traditional educational content overwhelming or inaccessible. The modularity of the platform further allows for easy future upgrades, while its cloud-based backend ensures scalability and data-driven intelligence.

However, the development journey came with its own set of challenges. Ensuring accuracy in AI-generated responses, maintaining performance across devices, and balancing personalization with general usability were significant hurdles. Overcoming these required continuous testing, fine-tuning AI prompt structures, optimizing the UI for clarity, and addressing data privacy concerns. Yet, these challenges provided valuable learning outcomes—especially in areas like prompt engineering, cloud integration, user-centric design, and scalable architecture.

Looking ahead, EduGenie holds immense potential for enhancement and expansion. One promising direction involves the integration of voice-based interaction, enabling users to learn hands-free using spoken commands and queries—a particularly useful feature for accessibility and multitasking. Multilingual support is another key frontier, helping bridge language barriers and reaching a broader global audience. A mobile

application is also envisioned to make learning available anytime, anywhere, with offline features for uninterrupted access.

Furthermore, future iterations of EduGenie could incorporate progress tracking dashboards, allowing users to visualize their learning journey through metrics and insights. The addition of gamified elements such as badges, learning streaks, and challenge-based assessments can boost engagement and motivation. Adaptive learning paths powered by data analytics can guide users more intelligently through content based on their strengths and weaknesses.

From a collaboration perspective, EduGenie can evolve to support group study sessions, teacher or parent dashboards, and integration with Learning Management Systems (LMS) such as Moodle or Google Classroom. Input recognition for images and PDF files can expand how students interact with content, enabling snapshot-based doubt solving and resource summarization. Real-time sync and smart notifications will also improve continuity and interactivity.

In conclusion, EduGenie is not just an AI tool—it's a foundation for the future of personalized, inclusive, and intelligent education. By bridging gaps in access, comprehension, and adaptability, it redefines the learning experience for digital natives and underserved communities alike. As it continues to evolve, EduGenie is poised to become a comprehensive learning companion for the modern era—smart, scalable, and learner-first.